

Fan Director Studio — Implementation Checklist

Complete step-by-step guide for developers to implement Fan Director Studio from design to production.

Project Overview

Fan Director Studio is a boutique creator commission platform with:

- **Fan Journey:** Select theme → AI Director conversation → Review scene card → Send to creator
- **Creator Dashboard:** Approve/decline requests, propose changes, ask questions
- **AI Director:** Mocked conversation system (no real AI in prototype)
- **Payment:** Not implemented (demo only)

Tech Stack:

- Frontend: React 18 + Vite + Tailwind CSS
 - Backend: Node.js + Express + TypeScript + PostgreSQL
 - Database: PostgreSQL 15 with Flyway migrations
 - DevOps: AWS (S3, CloudFront, ECS, RDS), Docker
 - CI/CD: GitHub Actions (6 workflows)
-

Phase 1: Local Setup (Day 1)

1.1 Clone Repository

```
git clone https://github.com/your-org/fan-director-studio.git
cd fan-director-studio
```

Acceptance Criteria:

- ☐ Repository cloned successfully
 - ☐ All branches visible (`main` , `develop`)
 - ☐ `.git/config` configured for your org
-

1.2 Environment Setup

Create `.env.local` in root:

```
# Database
DATABASE_URL=postgresql://postgres:password@localhost:5432/fan_director_dev
REDIS_URL=redis://localhost:6379

# Frontend
VITE_API_BASE_URL=http://localhost:3000/v1
VITE_ENVIRONMENT=development

# Backend
JWT_SECRET=dev-secret-key-change-in-production
ENVIRONMENT=development
LOG_LEVEL=debug
NODE_ENV=development

# AWS (for local testing - use IAM roles in production)
AWS_REGION=us-east-1
# Leave AWS keys blank for local dev (use localstack for S3)
```

Create `frontend/.env.local` :

```
VITE_API_BASE_URL=http://localhost:3000/v1
VITE_ENVIRONMENT=development
VITE_SENTRY_DSN=
```

Create `backend/.env.local` :

```
DATABASE_URL=postgresql://postgres:password@localhost:5432/fan_director_dev
REDIS_URL=redis://localhost:6379
JWT_SECRET=dev-secret-key
ENVIRONMENT=development
LOG_LEVEL=debug
PORT=3000
```

Acceptance Criteria:

- ☐ All `.env.local` files created
- ☐ No secrets committed to repo
- ☐ `.gitignore` includes `*.local`

1.3 Install Node & Package Manager

```
# Check Node version (18.x or 20.x)
node --version # v18.17.0 or higher
npm --version # 9.x or higher

# Install dependencies
npm install --workspaces

# Install global tools
npm install -g @flyway/cli newman
```

Acceptance Criteria:

- ☐ Node 18+ installed
- ☐ npm install completed without errors
- ☐ Flyway CLI available globally
- ☐ Newman CLI available globally

1.4 Start Docker Services

Create `docker-compose.yml` in root:

```
version: '3.9'
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
      POSTGRES_DB: fan_director_dev
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
```

```

localstack:
  image: localstack/localstack:latest
  environment:
    SERVICES: s3,cloudfront
    DEBUG: 1
  ports:
    - "4566:4566"
  volumes:
    - localstack_data:/tmp/localstack

volumes:
  postgres_data:
  localstack_data:

```

Start services:

```

docker-compose up -d
docker-compose ps # Verify all running

```

Acceptance Criteria:

- ☐ PostgreSQL running on port 5432
- ☐ Redis running on port 6379
- ☐ LocalStack running on port 4566
- ☐ All containers healthy

1.5 Initialize Database

```

# Create database schema
cd db
flyway migrate

# Verify schema
psql -h localhost -U postgres -d fan_director_dev -c "\dt"

```

Acceptance Criteria:

- ☐ Flyway migrations completed (V1__initial_schema.sql)
- ☐ All 8 tables created (`users` , `commission_requests` , `scene_cards` , etc.)
- ☐ Indexes created
- ☐ No migration errors

1.6 Seed Development Data

Run `db/seeds.sql`:

```
psql -h localhost -U postgres -d fan_director_dev -f db/seeds.sql
```

Expected data:

- ☐ 2 users created (1 fan: sarah@example.com, 1 creator: maya@example.com)
- ☐ 3 commission requests created
- ☐ 3 scene cards created
- ☐ Sample conversations populated

1.7 Start Backend Server

```
cd backend
npm run dev
```

```
# Expected output:
# ✓ Server running on http://localhost:3000
# ✓ Database connected
# ✓ Redis connected
```

Acceptance Criteria:

- ☐ Backend running on <http://localhost:3000>
- ☐ No console errors
- ☐ Database connection successful
- ☐ Health check returns 200: `curl http://localhost:3000/health`

1.8 Test API Endpoints

Quick health check:

```
curl -X POST http://localhost:3000/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"sarah@example.com","password":"password123"}'

# Expected response: 200 with JWT token
```

Acceptance Criteria:

- ☐ Login endpoint returns JWT token
 - ☐ Token is valid JSON Web Token
 - ☐ Can use token for authenticated requests
-

1.9 Start Frontend Development Server

```
cd frontend  
npm run dev
```

```
# Expected output:  
# ✓ VITE v4.x.x ready in xxx ms  
# ✓ Local: http://localhost:5173
```

Acceptance Criteria:

- ☐ Frontend running on <http://localhost:5173>
 - ☐ Hot module replacement (HMR) working
 - ☐ No console errors
 - ☐ Can load home page
-

1.10 Verify Full Stack Integration

Test user flow:

1. Navigate to <http://localhost:5173>
2. See Boutique Entrance page
3. Click "Start" button
4. Should navigate to theme selection
5. Network requests appear in DevTools

Acceptance Criteria:

- ☐ Frontend loads without errors
 - ☐ API calls working (check DevTools Network tab)
 - ☐ No CORS errors
 - ☐ Navigation between pages works
 - ☐ Auth token stored in localStorage
-

Phase 1 Summary

You've completed local setup when:

- ☐ Docker services running (PostgreSQL, Redis, LocalStack)
- ☐ Database schema initialized with migrations
- ☐ Backend server running on port 3000
- ☐ Frontend dev server running on port 5173
- ☐ Full-stack integration working (login, navigate, API calls)

Time estimate: 1-2 hours



Phase 2: Frontend Implementation (Days 2-4)

2.1 Review Design System

Study the Storybook (design_draft: 65e379f4-97fc-430a-941b-18d06b61f825):

- ☐ Color palette (8 colors with hex codes)
- ☐ Typography (Cormorant Garamond, Inter, JetBrains Mono)
- ☐ Button variants (primary, secondary, sizes)
- ☐ Input field styles (text, textarea, select)
- ☐ Status badges (4 types)
- ☐ Component patterns

Acceptance Criteria:

- ☐ Understand all color tokens
 - ☐ Know font families and sizes
 - ☐ Understand component API (props, states)
 - ☐ Can identify component variations
-

2.2 Set Up Component Library

Install dependencies:

```
cd frontend
npm install react-router-dom zustand axios
npm install -D tailwindcss postcss autoprefixer @testing-library/react vitest cypress
```

Create component structure:

```
mkdir -p src/{components,pages,hooks,context,utils,types,styles}

# Create component directories
mkdir -p src/components/{common,layout,forms,cards,modals}
```

Acceptance Criteria:

- ☐ All dependencies installed
- ☐ Directory structure created
- ☐ No installation errors

2.3 Implement Base Components

Priority 1 (Critical): Create these first

```
# Layout
src/components/layout/Header.jsx
src/components/layout/Footer.jsx

# Forms
src/components/forms/Input.jsx
src/components/forms/Textarea.jsx
src/components/forms/Select.jsx

# Common UI
src/components/common/Button.jsx
src/components/common/Badge.jsx
src/components/common/Modal.jsx

# Cards
src/components/cards/RequestCard.jsx
src/components/cards/SceneCard.jsx
```

Reference Storybook for exact styling. Example Button component:

```
// src/components/common/Button.jsx
export function Button({
  variant = 'primary',
  size = 'md',
  children,
  icon,
  disabled,
  ...props
}) {
  const baseClasses = 'rounded-[12px] font-medium transition-all focus-ring flex items-center justify-center gap-2'

  const variants = {
```

```

    primary: 'bg-rose text-espresso hover:bg-[#D595AE] shadow-sm',
    secondary: 'border border-divider bg-transparent text-espresso hover:bg-secondary'
  }

  const sizes = {
    sm: 'px-6 h-[44px] text-sm',
    md: 'px-8 h-[48px] text-base',
    lg: 'px-10 h-[52px] text-lg'
  }

  return (
    <button
      className={` ${baseClasses} ${variants[variant]} ${sizes[size]} ${disabled ? 'opacity-50 cursor-not-allowed' :
      disabled={disabled}
      {...props}
    >
      {children}
      {icon && <iconify-icon icon={icon} width="18"></iconify-icon>}
    </button>
  )
}

```

Acceptance Criteria:

- ☐ Button component renders correctly (all variants & sizes)
- ☐ Input fields match design specs (colors, padding, focus states)
- ☐ Modal can open/close
- ☐ RequestCard displays correctly
- ☐ SceneCard shows all sections
- ☐ Responsive on mobile/tablet/desktop

2.4 Implement Pages

Priority 1:

```

src/pages/BoutiqueEntrance.jsx
src/pages/AIDirector.jsx
src/pages/ReviewScene.jsx
src/pages/SendConfirmation.jsx

src/pages/CreatorDashboard.jsx
src/pages/CreatorDetailModal.jsx

```

Reference design pages:

- Boutique Entrance: 82277dd9-3db3-4c9d-8e8c-272cff83b5fe

- AI Director: 048b6b8a-1e95-4352-b0eb-32e252b1e30f
- Creator Dashboard: 2b65d4ed-45e0-464d-a22b-56a70377752b

Acceptance Criteria:

- ☐ All pages render without errors
 - ☐ Layout matches design (spacing, typography)
 - ☐ Navigation between pages works
 - ☐ Forms are functional (input, submit)
 - ☐ Data displays correctly
-

2.5 Set Up Routing

src/App.jsx:

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'
import { BoutiqueEntrance } from './pages/BoutiqueEntrance'
import { AIDirector } from './pages/AIDirector'
import { ReviewScene } from './pages/ReviewScene'
import { SendConfirmation } from './pages/SendConfirmation'
import { CreatorDashboard } from './pages/CreatorDashboard'

export function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<BoutiqueEntrance />} />
        <Route path="/ai-director" element={<AIDirector />} />
        <Route path="/review" element={<ReviewScene />} />
        <Route path="/confirmation" element={<SendConfirmation />} />
        <Route path="/creator/requests" element={<CreatorDashboard />} />
      </Routes>
    </BrowserRouter>
  )
}
```

Acceptance Criteria:

- ☐ All routes defined
 - ☐ Navigation works
 - ☐ Page transitions are smooth
 - ☐ URL updates correctly
-

2.6 Add State Management (Zustand)

src/store/authStore.js:

```
import { create } from 'zustand'

export const useAuthStore = create((set) => ({
  user: null,
  token: null,

  login: async (email, password) => {
    const res = await fetch('/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password })
    })
    const data = await res.json()
    set({ user: data.user, token: data.token })
  },

  logout: () => set({ user: null, token: null })
}))
```

Similar stores for:

- `requestStore` (commission requests)
- `sceneCardStore` (live scene card)
- `directorStore` (conversation state)

Acceptance Criteria:

- ☐ Auth state persists across page reloads
- ☐ Request data cached in store
- ☐ State updates trigger component re-renders

2.7 Integrate API Calls**src/api/client.js:**

```
import axios from 'axios'
import { useAuthStore } from '../store/authStore'

const client = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL
})

client.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token
  if (token) {
```

```

    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})

export default client

```

Create API modules:

- `api/auth.js` (login, register, logout)
- `api/requests.js` (CRUD operations)
- `api/director.js` (messages, choices)
- `api/creator.js` (approve, decline, ask)

Acceptance Criteria:

- ☐ All API calls use client instance
- ☐ JWT token auto-attached to requests
- ☐ Error handling implemented (401, 404, 500)
- ☐ Loading states managed

2.8 Add Form Validation

src/utils/validation.js:

```

export function validateEmail(email) {
  return /^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/ .test(email)
}

export function validatePassword(password) {
  return password.length >= 8
}

export function validateRequest(data) {
  const errors = {}
  if (!data.title) errors.title = 'Title required'
  if (data.budget <= 0) errors.budget = 'Budget must be positive'
  if (!data.requestedDeliveryDate) errors.date = 'Delivery date required'
  return errors
}

```

Acceptance Criteria:

- ☐ Form validation works on all forms
- ☐ Error messages display correctly
- ☐ Submit disabled until valid

- ☐ User feedback is clear

2.9 Implement Responsive Design

Test breakpoints:

- Mobile: 375px (iPhone SE)
- Tablet: 768px (iPad)
- Desktop: 1024px+ (MacBook)

Use Tailwind responsive classes:

```
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  { /* Mobile: 1 col, Tablet: 2 cols, Desktop: 3 cols */ }
</div>
```

Acceptance Criteria:

- ☐ All pages responsive at all breakpoints
- ☐ No horizontal scroll on mobile
- ☐ Touch targets 44px+ on mobile
- ☐ Typography scales appropriately

2.10 Add Loading & Error States

Loading skeleton:

```
export function RequestCardSkeleton() {
  return (
    <div className="animate-pulse">
      <div className="h-24 bg-secondary rounded-xl mb-4"></div>
    </div>
  )
}
```

Error boundary:

```
export class ErrorBoundary extends React.Component {
  componentDidCatch(error, info) {
    console.error('Error caught:', error, info)
  }

  render() {
```

```

    if (this.state.hasError) {
      return <div>Something went wrong. Please refresh.</div>
    }
    return this.props.children
  }
}

```

Acceptance Criteria:

- ☐ Loading states show while fetching
- ☐ Error states display gracefully
- ☐ User can retry failed requests
- ☐ No unhandled promise rejections

Phase 2 Summary

Frontend complete when:

- ☐ All 6 pages implemented and styled
- ☐ All components match Storybook reference
- ☐ API integration working
- ☐ Form validation implemented
- ☐ Responsive on all breakpoints
- ☐ Loading/error states handled

Time estimate: 3-5 days

Phase 3: Backend Implementation (Days 5-7)

3.1 Review API Specification

Study OpenAPI spec (from pinned notes):

- ☐ Understand all 17 endpoints
- ☐ Know request/response schemas
- ☐ Review authentication flow
- ☐ Understand error codes

3.2 Set Up Express Server

src/index.ts:

```
import express from 'express'
import cors from 'cors'
import { errorHandler } from './middleware/errorHandler'
import { authRoutes } from './routes/auth'
import { requestRoutes } from './routes/requests'

const app = express()

app.use(cors())
app.use(express.json())

// Routes
app.use('/v1/auth', authRoutes)
app.use('/v1/requests', requestRoutes)

// Error handling
app.use(errorHandler)

app.listen(3000, () => {
  console.log('Server running on port 3000')
})
```

Acceptance Criteria:

- ☐ Server starts without errors
- ☐ CORS enabled for frontend
- ☐ Request logging works
- ☐ Health check endpoint returns 200

3.3 Implement Authentication**routes/auth.ts:**

```
import { Router } from 'express'
import bcrypt from 'bcrypt'
import jwt from 'jsonwebtoken'
import { db } from '../db'

const router = Router()

// POST /v1/auth/register
router.post('/register', async (req, res) => {
  const { email, password, name, role } = req.body

  // Validate input
  // Hash password
```

```

const hashedPassword = await bcrypt.hash(password, 10)

// Insert user
const user = await db.query(
  'INSERT INTO users (email, password_hash, name, role) VALUES ($1, $2, $3, $4) RETURNING *',
  [email, hashedPassword, name, role]
)

res.status(201).json(user.rows[0])
})

// POST /v1/auth/login
router.post('/login', async (req, res) => {
  const { email, password } = req.body

  const user = await db.query('SELECT * FROM users WHERE email = $1', [email])
  if (!user.rows[0]) return res.status(401).json({ error: 'Invalid credentials' })

  const valid = await bcrypt.compare(password, user.rows[0].password_hash)
  if (!valid) return res.status(401).json({ error: 'Invalid credentials' })

  const token = jwt.sign({ userId: user.rows[0].id }, process.env.JWT_SECRET)

  res.json({ token, user: user.rows[0] })
})

export { router as authRoutes }

```

Acceptance Criteria:

- ☐ User registration works
- ☐ User login returns JWT
- ☐ Password hashing implemented
- ☐ Token validation middleware works

3.4 Implement Request Endpoints

routes/requests.ts:

```

// GET /v1/requests (list user requests)
// POST /v1/requests (create new request)
// GET /v1/requests/:id (get details)
// PATCH /v1/requests/:id (update request)

// Implementation pattern:
router.post('/requests', authRequired, async (req, res) => {
  const { title, budget, requestedDeliveryDate } = req.body
  const userId = req.user.id

```

```
const request = await db.query(
  'INSERT INTO commission_requests (...) VALUES (...) RETURNING *',
  [userId, title, budget, requestedDeliveryDate]
)

res.status(201).json(request.rows[0])
})
```

Acceptance Criteria:

- ☐ CRUD operations work for requests
- ☐ Auth required on all endpoints
- ☐ Status filtering works
- ☐ Pagination implemented

3.5 Implement Director Endpoints

routes/director.ts:

```
// POST /v1/director/message (send message)
// POST /v1/director/choice (select choice)
// GET /v1/scene/:requestId (get live scene)

// For now, responses are mocked (no real AI)
router.post('/director/message', authRequired, async (req, res) => {
  const { requestId, message } = req.body

  // Save conversation
  await db.query(
    'INSERT INTO director_conversations (request_id, sender, content) VALUES ($1, $2, $3)',
    [requestId, 'fan', message]
  )

  // Mock response (replace with real AI later)
  const mockResponse = {
    message: "Great! Let's create something special...",
    sceneCard: { /* updated scene */ },
    suggestions: ["Option A", "Option B"]
  }

  res.json(mockResponse)
})
```

Acceptance Criteria:

- ☐ Messages saved to database
- ☐ Scene card updates

- ☐ Mock responses consistent
- ☐ Conversation history retrievable

3.6 Implement Creator Endpoints

routes/creator.ts:

```
// GET /v1/creator/requests (list creator's requests)
// GET /v1/creator/requests/:id (get detail)
// POST /v1/creator/requests/:id/approve
// POST /v1/creator/requests/:id/decline
// POST /v1/creator/requests/:id/ask
// POST /v1/creator/requests/:id/propose

// Example:
router.post('/:id/approve', creatorRequired, async (req, res) => {
  const { id } = req.params
  const { notes } = req.body
  const creatorId = req.user.id

  // Update request status
  const updated = await db.query(
    'UPDATE commission_requests SET status = $1, approved_at = NOW() WHERE id = $2 RETURNING *',
    ['approved', id]
  )

  // Save creator decision
  await db.query(
    'INSERT INTO creator_decisions (request_id, creator_id, decision_type, notes) VALUES ($1, $2, $3, $4)',
    [id, creatorId, 'approve', notes]
  )

  res.json(updated.rows[0])
})
```

Acceptance Criteria:

- ☐ All 6 creator actions work
- ☐ Status transitions correct
- ☐ Decisions saved to audit log
- ☐ Timestamps recorded

3.7 Add Middleware

middleware/auth.ts:

```
import jwt from 'jsonwebtoken'

export function authRequired(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1]
  if (!token) return res.status(401).json({ error: 'Unauthorized' })

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET)
    req.user = decoded
    next()
  } catch {
    res.status(401).json({ error: 'Invalid token' })
  }
}

export function creatorRequired(req, res, next) {
  if (req.user.role !== 'creator') {
    return res.status(403).json({ error: 'Forbidden' })
  }
  next()
}
```

Acceptance Criteria:

- ☐ Auth middleware validates tokens
- ☐ Role-based access control works
- ☐ Unauthorized requests rejected
- ☐ Error messages clear

3.8 Add Request Validation

middleware/validate.ts:

```
export function validateRequest(schema) {
  return (req, res, next) => {
    const { error, value } = schema.validate(req.body)
    if (error) {
      return res.status(400).json({ error: error.details[0].message })
    }
    req.validated = value
    next()
  }
}

// Usage:
router.post('/requests', validateRequest(createRequestSchema), async (req, res) => {
```

```
// req.validated has validated data
})
```

Acceptance Criteria:

- ☐ Input validation on all endpoints
 - ☐ Invalid data rejected
 - ☐ Error messages helpful
 - ☐ No SQL injection possible
-

3.9 Add Database Connection Pooling

db/index.ts:

```
import { Pool } from 'pg'

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 20,
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
})

pool.on('error', (err) => {
  console.error('Unexpected error on idle client', err)
})

export const db = {
  query: (text, params) => pool.query(text, params),
  end: () => pool.end()
}
```

Acceptance Criteria:

- ☐ Connection pooling configured
 - ☐ Max 20 connections
 - ☐ Proper error handling
 - ☐ Graceful shutdown
-

3.10 Add Logging & Monitoring

middleware/logger.ts:

```
import winston from 'winston'

export const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.json(),
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' })
  ]
})

// Middleware
export function requestLogger(req, res, next) {
  logger.info({
    method: req.method,
    path: req.path,
    ip: req.ip,
    timestamp: new Date().toISOString()
  })
  next()
}
```

Acceptance Criteria:

- ☐ All requests logged
- ☐ Error logs in separate file
- ☐ Log level configurable
- ☐ Sensitive data not logged

Phase 3 Summary

Backend complete when:

- ☐ All 17 API endpoints implemented
- ☐ Authentication working (JWT)
- ☐ Creator/fan role-based access control
- ☐ Request validation on all endpoints
- ☐ Database queries optimized
- ☐ Logging implemented
- ☐ Error handling comprehensive

Time estimate: 3-5 days



Phase 4: Testing (Days 8-9)

4.1 Unit Tests (Frontend)

```
# src/components/__tests__/Button.test.jsx
import { render, screen } from '@testing-library/react'
import { Button } from '../Button'

describe('Button', () => {
  it('renders primary button', () => {
    render(<Button variant="primary">Click me</Button>)
    expect(screen.getByText('Click me')).toBeInTheDocument()
  })

  it('disables button when disabled prop is true', () => {
    render(<Button disabled>Click me</Button>)
    expect(screen.getByRole('button')).toBeDisabled()
  })
})
```

Run tests:

```
cd frontend
npm run test:unit -- --coverage
```

Target:

- ☐ 80%+ code coverage
- ☐ All components tested
- ☐ Props validation tested
- ☐ User interactions tested

4.2 Integration Tests (Frontend)

```
# Test full user flow
describe('Fan Journey', () => {
  it('completes commission request', () => {
    cy.visit('http://localhost:5173')
    cy.contains('Start').click()
    cy.url().should('include', '/ai-director')
    // ... continue user flow
  })
})
```

Run tests:

```
cd frontend
npm run test:e2e
```

Acceptance Criteria:

- ☐ All pages accessible
- ☐ User flows complete
- ☐ Navigation works
- ☐ No console errors

4.3 Unit Tests (Backend)

```
// src/__tests__/auth.test.ts
import request from 'supertest'
import app from '../app'

describe('Auth Endpoints', () => {
  it('POST /auth/login returns token', async () => {
    const res = await request(app)
      .post('/v1/auth/login')
      .send({ email: 'test@example.com', password: 'password' })

    expect(res.status).toBe(200)
    expect(res.body.token).toBeDefined()
  })
})
```

Run tests:

```
cd backend
npm run test:unit -- --coverage
```

Target:

- ☐ 80%+ code coverage
- ☐ All endpoints tested
- ☐ Error cases tested
- ☐ Database queries tested

4.4 API Tests (Postman)

Use provided Postman collection:

```
newman run postman_collection.json \
  -e environments/development.json \
  --reporters cli,json
```

Test scenarios:

- ☐ Complete fan journey
- ☐ Complete creator review
- ☐ All error cases
- ☐ Authentication flow

4.5 Performance Tests

```
# Lighthouse
npm run lighthouse

# API response times
# Should be < 200ms for most endpoints
```

Acceptance Criteria:

- ☐ Frontend Lighthouse score > 90
- ☐ API response times < 200ms
- ☐ Database queries optimized
- ☐ No memory leaks

✓ Phase 4 Summary

Testing complete when:

- ☐ Frontend unit tests: 80%+ coverage
- ☐ Frontend E2E tests: All flows passing
- ☐ Backend unit tests: 80%+ coverage
- ☐ API tests: All endpoints passing
- ☐ Performance tests: All metrics passing

Time estimate: 2 days

Phase 5: Deployment (Days 10-11)

5.1 Configure GitHub Secrets

Go to **Settings** → **Secrets and variables** → **Actions**:

```
AWS_ACCESS_KEY_ID=***
AWS_SECRET_ACCESS_KEY=***
AWS_ACCOUNT_ID=123456789
STAGING_CLOUDFRONT_DIST_ID=***
PROD_CLOUDFRONT_DIST_ID=***
SNYK_TOKEN=***
SENTRY_AUTH_TOKEN=***
SLACK_WEBHOOK_STAGING=***
SLACK_WEBHOOK_PROD=***
```

Acceptance Criteria:

- ☐ All 11 secrets configured
- ☐ No secrets in code
- ☐ GitHub Actions can access secrets

5.2 Configure Branch Protection

Settings → **Branches** → **Add rule** for `main` and `develop`:

- ☒ Require a pull request before merging
- ☒ Require status checks to pass before merging
- ☒ Require code reviews before merging (1 approved)
- ☒ Require administrators to follow the same rules

Acceptance Criteria:

- ☐ Branch protection enabled
- ☐ CI/CD checks required
- ☐ Code review required

5.3 Deploy to Staging

```
git checkout develop
git pull origin develop

# CI runs automatically:
# 1. Frontend CI (lint, test, build)
# 2. Backend CI (lint, test, build)
# 3. Deploy to Staging (S3, CloudFront, ECS)
```

```
# Monitor: GitHub Actions tab
# URL: https://staging.fandirectorstudio.com
```

Acceptance Criteria:

- ☐ All CI checks pass
- ☐ Frontend deployed to S3
- ☐ Backend deployed to ECS
- ☐ Smoke tests pass
- ☐ Slack notification received

5.4 Test in Staging

```
# Run full Postman collection
newman run postman_collection.json \
  -e environments/staging.json

# Test user flows manually
# Test creator workflows manually
```

Acceptance Criteria:

- ☐ All API endpoints working
- ☐ Fan journey complete
- ☐ Creator workflows working
- ☐ No errors in logs

5.5 Deploy to Production

```
# Create GitHub release
git tag v1.0.0
git push origin v1.0.0

# OR merge main branch
git checkout main
git merge develop
git push origin main

# CI/CD runs:
# 1. Database migrations
# 2. Frontend build → S3 + CloudFront
# 3. Backend build → Docker → ECS
```

- # 4. Smoke tests
- # 5. Sentry release + Slack alerts

Acceptance Criteria:

- ☐ All CI checks pass
 - ☐ Database migrated successfully
 - ☐ Zero-downtime deployment
 - ☐ Smoke tests pass
 - ☐ Stakeholders notified
-

5.6 Monitor Production

Set up monitoring:

- ☐ CloudWatch logs configured
- ☐ Sentry error tracking enabled
- ☐ Slack alerts for errors
- ☐ Uptime monitoring (Pingdom/StatusPage)

Acceptance Criteria:

- ☐ Can view logs in CloudWatch
 - ☐ Errors appear in Sentry
 - ☐ Alert notifications working
 - ☐ Uptime dashboard live
-

Phase 5 Summary

Deployment complete when:

- ☐ Staging environment working
- ☐ All tests passing
- ☐ Production deployed
- ☐ Monitoring configured
- ☐ Team trained on deployment

Time estimate: 1 day



Implementation Timeline

Phase	Duration	Status
1. Local Setup	1-2 days	✓
2. Frontend	3-5 days	⌚
3. Backend	3-5 days	⌚
4. Testing	2 days	⌚
5. Deployment	1 day	⌚
Total	10-15 days	



Key Milestones

- **Day 1:** Full-stack running locally ✓
- **Day 4:** Frontend pages implemented
- **Day 7:** All API endpoints working
- **Day 9:** Tests passing, 80%+ coverage
- **Day 10:** Staging environment live
- **Day 11:** Production deployment



Reference Documents

All pinned in project:

1. **Design System** — Colors, fonts, spacing, components
2. **Developer Handoff** — Tech stack, file structure, JSX examples
3. **API Specification** — OpenAPI 3.0 with all endpoints
4. **Database Schema** — PostgreSQL with migrations
5. **Postman Collection** — API testing with examples
6. **GitHub Actions** — CI/CD workflows



Troubleshooting

Database connection fails

```
# Check PostgreSQL running
docker-compose ps postgres

# Check credentials in .env
echo $DATABASE_URL

# Reset database
dropdb fan_director_dev
createdb fan_director_dev
flyway migrate
```

Frontend can't reach API

```
# Check backend running
curl http://localhost:3000/health

# Check CORS
# Should see 'Access-Control-Allow-Origin' header

# Check API URL in .env
echo $VITE_API_BASE_URL
```

Tests failing

```
# Clear cache
rm -rf node_modules/.cache

# Reinstall dependencies
npm ci

# Run with verbose output
npm run test:unit -- --verbose
```

Learning Resources

- [React Documentation](#)
- [Express Guide](#)
- [PostgreSQL Docs](#)
- [GitHub Actions](#)
- [Tailwind CSS](#)

Success Criteria

You've successfully implemented Fan Director Studio when:

✓ Local Development

- ☐ All services running (PostgreSQL, Redis, Backend, Frontend)
- ☐ Can access <http://localhost:5173>
- ☐ Can log in and navigate

✓ Frontend Complete

- ☐ All 6 pages implemented and styled
- ☐ Responsive on all breakpoints
- ☐ API integration working
- ☐ Unit tests: 80%+ coverage

✓ Backend Complete

- ☐ All 17 API endpoints working
- ☐ JWT authentication working
- ☐ Role-based access control
- ☐ Unit tests: 80%+ coverage

✓ Deployment Ready

- ☐ Staging environment live
- ☐ Production deployment successful
- ☐ CI/CD pipelines working
- ☐ Monitoring configured

✓ Team Ready

- ☐ All developers trained
- ☐ Documentation accessible
- ☐ Runbooks created
- ☐ Escalation paths clear

Congratulations! 🎉 Your Fan Director Studio implementation is complete and ready for production.